

PRIDE Vision AI

Projeto Simplificado e Alinhado a ASPM

1. Ideia do projeto

O **PRIDE Vision AI** será uma plataforma simplificada de **ASPM (Application Security Posture Management)**.

A plataforma ajudará uma empresa a:

- Cadastrar suas aplicações e ativos.
- Receber resultados de segurança.
- Organizar vulnerabilidades.
- Comparar resultados do **Semgrep** e do **Nuclei**.
- Priorizar os problemas mais importantes.
- Acompanhar o status da correção.
- Usar **IA** para explicar a vulnerabilidade e sugerir uma solução.

O objetivo não é criar outro scanner. O **PRIDE** vai funcionar como uma central de organização, priorização e acompanhamento dos riscos de segurança da aplicação, que é a ideia principal de uma solução **ASPM**.

2. Problema

As ferramentas de segurança geram vários alertas, mas nem todos possuem o mesmo nível de importância.

Por exemplo:

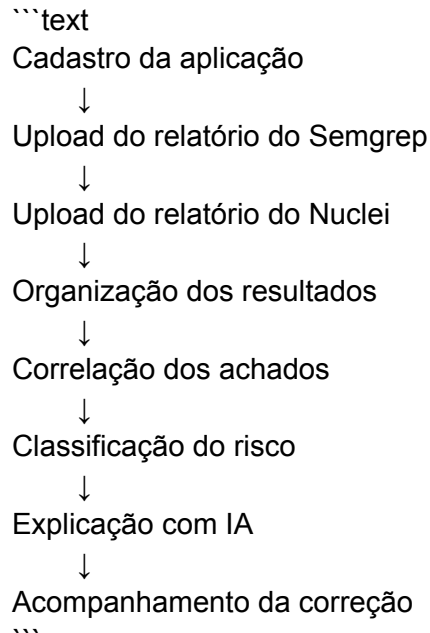
- O **Semgrep** pode encontrar uma possível falha no código.
- O **Nuclei** pode confirmar que existe um comportamento vulnerável na aplicação.
- Uma vulnerabilidade em **produção** pode ser mais importante que a mesma falha em um ambiente de teste.

O **PRIDE** cruza essas informações e ajuda a equipe a responder:

> “Qual vulnerabilidade precisa ser corrigida primeiro?”

3. Como o sistema funcionará

O sistema terá este fluxo:



A primeira versão receberá os arquivos **manualmente**. O sistema não executará automaticamente o Semgrep ou o Nuclei.

4. O que significa ASPM neste projeto

Para o projeto não ficar apenas como um “comparador de arquivos”, ele terá quatro partes principais:

1. Inventário de aplicações

O usuário poderá cadastrar:

- Nome da aplicação.
- Responsável.
- Ambiente.
- URL.
- Exposição externa ou interna.
- Importância para o negócio.

Exemplo:

```text

Aplicação: Portal do Cliente

Ambiente: Produção

Exposição: Internet

Importância: Alta

```

2. Centralização dos achados

Os resultados do Semgrep e do Nuclei ficarão reunidos em uma única tela.

Isso permitirá visualizar:

- Qual ferramenta encontrou o problema.
- Qual aplicação foi afetada.
- Qual o tipo da vulnerabilidade.
- Qual a severidade original.
- Qual o status da análise.

3. Priorização baseada em risco

O sistema calculará uma prioridade usando:

- Severidade indicada pela ferramenta.
- Confirmação pelo outro scanner.
- Ambiente da aplicação.
- Exposição da aplicação.
- Importância para o negócio.

4. Acompanhamento da correção

Cada vulnerabilidade terá um status:

- Nova / Em análise.
- Em análise.
 - Em correção / Corrigida.
- Corrigida.
 - Falso positivo.

Essa parte é importante para o conceito de ASPM, pois a plataforma não apenas mostra vulnerabilidades: ela também ajuda a acompanhar o ciclo de tratamento.

5. Regra de risco

A regra será simples e explicável.

Risco Crítico

Quando:

- Semgrep e Nuclei confirmam a falha em **produção**.
- O Nuclei encontra evidência relacionada.
- A aplicação está em produção.
- A aplicação está exposta à internet ou possui alta importância.

Risco Alto

Quando:

- Vulnerabilidade relevante no código em **produção**, sem confirmação dinâmica.
- A aplicação está em produção.
- Ainda não há confirmação pelo Nuclei.

Risco Médio/Baixo

Quando:

- Falhas em ambientes de homologação ou teste com baixo impacto.
- A aplicação está em homologação.
- Não existe evidência clara de exploração.

Risco baixo

Quando:

- A aplicação está em ambiente de teste.
- O problema possui baixo impacto.
- O achado parece ser apenas teórico.

A IA não será responsável por definir o risco. A classificação será feita pelo sistema usando regras fixas. Isso deixa o resultado mais fácil de explicar, testar e auditar.
[paloaltonetworks](https://www.paloaltonetworks.com/cyberpedia/application-security-best-practices)

6. Como funcionará a IA

A IA será um ****assistente de segurança****, não o cérebro principal do sistema.

A **IA** atuará como um assistente, mascarando informações sensíveis (IPs, tokens, chaves) antes da análise.

A IA poderá:

- Explicar vulnerabilidades e sugerir remediações.
- Informar o possível impacto.
- Explicar por que o risco foi priorizado.
 - Criar descrições para tickets de correção.
- Criar uma descrição para um ticket.

A IA não poderá:

- Alterar o código automaticamente.
- Fazer commit.
- Aprovar a correção sozinha.
- Mudar a classificação sem autorização.
- Executar ataques.

Antes de enviar dados para a IA, o sistema mascarará informações sensíveis, como:

- IPs.
- Tokens.
- Senhas.
- Chaves de API.
- Nomes internos.

Exemplo:

```
```text
IP real: 192.168.0.10
Enviado para a IA: [IP_MASCARADO]
```
```

7. Interface em React

7. Interface e Tecnologias

A comunicação entre React e o backend será feita por uma API.

Tela inicial — Visão geral

Mostrará:

- Total de aplicações cadastradas.
- Total de vulnerabilidades.
- Vulnerabilidades críticas.
- Vulnerabilidades em correção.
- Aplicações com maior risco.

Tela de aplicações

Mostrará:

- Lista de aplicações.
- Ambiente.
- Exposição.
- Importância para o negócio.
- Quantidade de vulnerabilidades.

Tela de vulnerabilidades

Mostrará:

- Nome da vulnerabilidade.
- Aplicação afetada.
- Origem: Semgrep ou Nuclei.
- Severidade.
- Risco calculado pelo PRIDE.
- Status da correção.
- Data da identificação.

Tela de detalhes

Mostrará:

- Explicação da vulnerabilidade.
- Arquivo e linha, quando disponíveis.
- Endpoint ou URL afetada.
- Evidência do Nuclei.

- Justificativa da classificação.
- Sugestão da IA.
- Botão para alterar o status.

8. Tecnologias

Frontend

- **Frontend:** React, Vite, TypeScript, Tailwind CSS.
- Vite
- TypeScript
- Tailwind CSS
- React Router
- Axios ou React Query
- Recharts para gráficos simples

Backend

- **Backend:** Python, FastAPI, SQLAlchemy, SQLite.
- FastAPI
- Pydantic
- SQLite na POC
- SQLAlchemy
- API REST

Como o frontend será React, precisaremos de um backend com API. Por isso, o FastAPI volta ao projeto, mas apenas com uma função simples: receber dados, aplicar as regras, salvar no banco e responder ao React.

IA

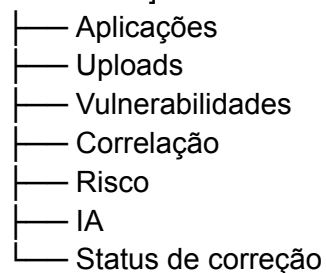
- **IA:** OpenAI ou Gemini API.
- Prompt fixo.
- Dados mascarados antes do envio.

9. Arquitetura simplificada

[React]



[API FastAPI]



[SQLite]

A arquitetura ainda será um monólito, ou seja, um único backend organizado por módulos. Não serão usados microserviços, Kubernetes ou infraestrutura complexa.

10. Escopo do MVP

O MVP precisa ter somente:

- Login simples.
- Cadastro de aplicações.
- Upload de `semgrep.json`.
- Upload de `nuclei.jsonl`.
- Leitura e organização dos dados.
- Comparação entre os resultados.
- Classificação de risco.
- Lista de vulnerabilidades.
- Alteração do status de correção.
- Dashboard simples.
- Explicação por IA com dados mascarados.

Esse conjunto já demonstra as principais características de um ASPM: inventário, centralização, priorização e acompanhamento.

12. Divisão do grupo

Pessoa 1 — Backend e ingestão

Responsável por:

- Criar a API com FastAPI.
- Fazer o upload dos arquivos.
- Criar o parser do Semgrep.
- Criar o parser do Nuclei.
- Padronizar os dados.

Pessoa 2 — Correlação e risco

Responsável por:

- Comparar os resultados.
- Definir o que é um match.
- Criar a matriz de risco.
- Criar o cadastro de aplicações.
- Implementar o status das vulnerabilidades.

Pessoa 3 — IA e banco de dados

Responsável por:

- Criar as tabelas do SQLite.
- Salvar as aplicações e vulnerabilidades.
- Criar o mascaramento de dados.
- Criar o prompt da IA.
- Integrar a API do modelo de IA.
- Criar o layout.
- Criar as telas.
- Criar tabelas e filtros.
- Criar o dashboard.
- Conectar o React com a API.

13. Roadmap de produção

Etapa 1 — Preparar o projeto

- Criar os repositórios frontend e backend.
- Definir os campos dos dados.
- Criar arquivos de exemplo.
- Definir as telas principais.

Etapa 2 — Criar o backend básico

- Criar a API FastAPI.
- Criar o banco SQLite.
- Criar cadastro de aplicações.
- Criar upload dos arquivos.

Etapa 3 — Criar os parsers

- Ler o arquivo do Semgrep.
- Ler o arquivo do Nuclei.
- Transformar os dados no padrão PRIDE.
- Salvar os findings no banco.

Etapa 4 — Criar o motor de risco

- Comparar vulnerabilidades.
- Identificar matches.
- Considerar ambiente e exposição.
- Gerar classificação final.

Etapa 5 — Criar o frontend

- Criar login.
- Criar dashboard.

- Criar tela de aplicações.
- Criar tela de vulnerabilidades.
- Criar tela de detalhes.

Etapa 6 — Integrar a IA

- Criar o masking.
- Criar o prompt.
- Enviar apenas informações sanitizadas.
- Exibir explicação e remediação.

Etapa 7 — Criar o acompanhamento

- Adicionar status da vulnerabilidade.
- Permitir marcar como em correção.
- Permitir marcar como corrigida.
- Exibir quantidade por status.

Etapa 8 — Integração final

- Conectar todas as telas à API.
- Testar o fluxo completo.
- Corrigir erros.
- Melhorar a interface.
- Testar com diferentes arquivos.

14. Definição final do projeto

> O PRIDE Vision AI é uma plataforma simplificada de ASPM que centraliza informações de segurança de aplicações, correlaciona resultados do Semgrep e do Nuclei, prioriza vulnerabilidades de acordo com o risco técnico e o contexto do negócio, e utiliza inteligência artificial para explicar os problemas e sugerir correções. A plataforma também permite acompanhar o status de tratamento de cada vulnerabilidade.

Essa versão mantém a ideia original de ASPM, mantém a inteligência artificial e utiliza React, mas evita que o grupo precise construir uma plataforma empresarial completa. O foco será entregar um fluxo funcional:



Remediação